```
    dz = (vmax-vmin)/(nsup-1.)
    target_dist_batch = dist_batch.clone()
    for i in range(dist_batch.shape[0]):
1
        dist_full = dist_batch[i]
        action = int(action_batch[i].item())
        dist = dist_full[action]
        r = reward_batch[i]
        if r != -1:
2
            target_dist = torch.zeros(nsup)
            bj = np.round((r-vmin)/dz)
            bj = int(np.clip(bj,0,nsup-1))
            target_dist[bj] = 1.
        else:
3
            target_dist =
update_dist(r,support,dist,lim=lim,gamma=gamma)
        target_dist_batch[i,action,:] = target_dist
4

    return target_dist_batch
```

- ***1*** Loops through the batch dimension
- ***2*** If the reward is not –1, it is a terminal state and the target is a degenerate distribution at the reward value.
- ***3*** If the state is nonterminal, the target distribution is a Bayesian update of the prior given the reward.
- ***4*** Only changes the distribution for the action that was taken

The `get_target_dist` function takes a batch of data of shape $B \times 3 \times 51$ where B is the batch dimension, and it returns an equal-sized tensor. For example, if we only have one example in our batch, $1 \times 3 \times 51$, and the agent took action 1 and observed a reward of –1, this function would return a $1 \times 3 \times 51$ tensor, except that the 1×51 distribution associated with index 1 (of dimension 1) will be changed according to the `update_dist` function using the observed reward of –1. If the observed reward was instead 10, the 1×51 distribution associated with action 1 would be updated to be a degenerate distribution where all elements have 0 probability except the one associated with the reward of 10 (index 50).

7.5. Comparing probability distributions